

C++ STUDY NOTES

CHAPTER 2

Classes

Data Members, Member Functions, Friend Functions, Static Members, Constructors, Destructors, Object Arrays, Function & Operator Overloading

University : Rani Channamma University, Belagavi

Course Code : 24MSCCS 1.3

Programme : M.Sc. Computer Science

Semester : First Semester

Syllabus : 2024-25 onwards

Author : Sudarshan Badli

Academic Yr : 2026-2027

AUTHORSHIP

Prepared and Created by: Sudarshan Badli

© 2026 Sudarshan Badli. All Rights Reserved.

Table of Contents

2.1 Class Definition, Data Members, Member Functions, Access	3
2.2 Class Declaration vs Class Definition; Defining Member Functions Outside the Class	4
2.3 Friend Functions	5
2.4 Class Object Sets, Class Member Functions, The Implicit this Pointer	6
2.5 Static Class Members	7
2.6 Pointer to Class Member	8
2.7 Nested Classes	9
2.8 Class Initialization, Member-wise Initialization and Member-wise Assignment	10
2.9 The Class Constructor	11
2.10 The Class Destructor	13
2.11 Class Object Arrays	14
2.12 Function Overloading	15
2.13 Operator Overloading	16
2.20 Examination Preparation — Chapter 2	20

HOW THIS CHAPTER IS ORGANISED

- Follows the official RCUB M.Sc. Computer Science syllabus (24MSCCS 1.3), Unit II, exactly — no topic is skipped or compressed.
- Each topic: Definition → Syntax → Key points → Full worked C++ program(s) with output.
- Sections 2.14–2.19 add deep-dive material: dry runs/trace tables, line-by-line explanation, advantages/disadvantages/applications, common mistakes, the Rule of Three, and head-to-head comparisons.
- Section 2.20 is a complete examination-preparation pack: 2/5/10-mark Q&A, programming questions, output prediction, find-the-error, MCQs and viva questions.

2.
1

Class Definition, Data Members, Member Functions, Access

DEFINITION

A **class** bundles *data members* (attributes/state) and *member functions* (behaviour) under one name. **Access specifiers** — `private`, `protected`, `public` — control which code can reach a member: `private`/`protected` members are reachable only from inside the class (`protected` also from derived classes, Unit III), `public` members are reachable from anywhere the object is visible.

SYNTAX

```
class ClassName {  
    private:  
        dataType member1;           // hidden (default access in a class)  
    protected:  
        dataType member2;           // visible to derived classes too  
    public:  
        dataType member3;  
        returnType memberFunction(parameters);  
};
```

KEY POINTS

- Data hiding: keeping data members `private` and exposing controlled access through `public` *getter/setter* functions is the standard encapsulation pattern.
- A member function may be defined inside the class body (implicitly `inline`) or outside it using the scope resolution operator `::`.
- Member access from outside uses the dot operator: `object.member`.

PROGRAM – GETTERS/SETTERS

```
#include <iostream>  
using namespace std;  
  
class Student {  
    private:  
        int rollNo;  
        string name;  
        float marks;  
    public:  
        void setData(int r, string n, float m) {  
            rollNo = r; name = n; marks = m;  
        }  
        void display() {  
            cout << rollNo << " " << name << " " << marks << "\n";  
        }  
};  
  
int main() {  
    Student s;  
    s.setData(101, "Asha", 88.5);  
    s.display();  
    return 0;  
}
```

OUTPUT

```
101  Asha  88.5
```

2. Class Declaration vs Class Definition; Defining Member Functions Outside the Class

DEFINITION

A **class declaration** (also called the class *interface*) lists the members' names and types without necessarily giving the function bodies. The **class definition** supplies full implementations. Splitting them (declaration in a .h file, definitions in a .cpp file) is standard practice for larger programs; a member function defined outside the class body needs the scope resolution operator `ClassName::`.

SYNTAX

```
class Box {
public:
    double volume();           // declaration only (prototype)
private:
    double l, b, h;
};

// definition outside the class
double Box::volume() {
    return l * b * h;
}
```

KEY POINTS

- The scope resolution operator `::` tells the compiler which class the function belongs to.
- This separation supports encapsulation at the file level: users of the class see only the header (what it does), not the implementation (how).

PROGRAM

```
#include <iostream>
using namespace std;

class Box {
private:
    double l, b, h;
public:
    void setDims(double a, double bb, double c) { l = a; b = bb; h = c; }
    double volume();           // declared here
};

double Box::volume() {         // defined outside, using ::
    return l * b * h;
}

int main() {
    Box box;
    box.setDims(2, 3, 4);
    cout << "Volume = " << box.volume() << "\n";
    return 0;
}
```

OUTPUT

```
Volume = 24
```

2.3 Friend Functions

DEFINITION

A **friend function** is a non-member function that is explicitly granted access to a class's private and protected members. It is declared inside the class with the `friend` keyword but defined outside like a normal function (it is **not** a member, so it is not called with the dot operator and has no `this` pointer).

SYNTAX

```
class ClassName {  
    friend returnType functionName(parameters); // declaration  
    ...  
};  
  
returnType functionName(parameters) { // ordinary function  
    // can access private members of ClassName objects passed in  
}
```

KEY POINTS

- Friendship is granted by the class, not taken by the function; it is not inherited and not mutual (A befriending B does not make B able to befriend A automatically).
- Common use: operator overloading (e.g. `operator<<` for printing) and functions that need to operate on two different classes' private data (e.g. comparing a Meters and a Feet object).
- A whole class can also be declared a friend of another class: `friend class OtherClass;`

PROGRAM – COMPARING PRIVATE DATA OF TWO OBJECTS

```
#include <iostream>  
using namespace std;  
  
class Box {  
private:  
    double volume;  
public:  
    Box(double v) : volume(v) {}  
    friend Box bigger(const Box &a, const Box &b); // friend  
};  
  
Box bigger(const Box &a, const Box &b) {  
    return (a.volume > b.volume) ? a : b; // accesses private volume  
}  
  
int main() {  
    Box b1(24), b2(40);  
    Box big = bigger(b1, b2);  
    cout << "Larger volume = " << big.volume << "\n";  
    return 0;  
}
```

OUTPUT

```
Larger volume = 40
```

KEY POINTS

- Overusing friend functions weakens encapsulation; use them only when genuinely needed (e.g. symmetric operators, I/O overloading).

2. Class Object Sets, Class Member Functions, The Implicit `this` Pointer

DEFINITION

Multiple objects of the same class (a "class object set") each get their own copy of the data members but share one copy of the member-function code. Every non-static member function secretly receives a hidden first parameter, the **this** pointer, which points to the specific object the function was called on — this is how the same function body operates on different objects' data.

SYNTAX

```
class Counter {
    int value;
public:
    void setValue(int value) {
        this->value = value;    // "this" disambiguates member vs parameter
    }
    Counter* getAddress() { return this; }
};
```

KEY POINTS

- `this` is a pointer to the current object (`ClassName *const this`), available inside every non-static member function.
- Typical uses: resolving a naming clash between a parameter and a data member, returning `*this` to enable function chaining, and passing the current object's address to another function.

PROGRAM – THIS POINTER AND NAME CLASH

```
#include <iostream>
using namespace std;

class Point {
    int x, y;
public:
    void setXY(int x, int y) {
        this->x = x;    // this->x is the member; x is the parameter
        this->y = y;
    }
    void show() { cout << "(" << x << "," << y << ")\n"; }
};

int main() {
    Point p1, p2;
    p1.setXY(3, 4);
    p2.setXY(7, 9);    // same function, different objects
    p1.show(); p2.show();
    return 0;
}
```

OUTPUT

```
(3,4)
(7,9)
```

PROGRAM – METHOD CHAINING USING *THIS

```
#include <iostream>
using namespace std;

class Builder {
    string text;
public:
    Builder& add(string s) { text += s; return *this; }    // returns the object itself
    void print() { cout << text << "\n"; }
};

int main() {
    Builder b;
    b.add("Hello, ").add("World").add("!").print();
    return 0;
}
```

OUTPUT

```
Hello, World!
```

2.5 Static Class Members

DEFINITION

A **static data member** is shared by *all* objects of the class — only one copy exists regardless of how many objects are created, useful for counters or shared configuration. A **static member function** can access only static data (it has no *this* pointer) and can be called using the class name, without an object.

SYNTAX

```
class ClassName {
    static dataType member;           // declaration inside the class
public:
    static returnType staticFn();
};
dataType ClassName::member = initialValue;    // definition outside (once)
```

KEY POINTS

- A static data member must be defined (and usually initialised) exactly once outside the class, at file/global scope.
- Static members belong to the class itself, not to any one object — accessible as `ClassName::member` even with zero objects created.
- Common use: counting how many objects currently exist.

PROGRAM – OBJECT COUNTER

```
#include <iostream>
using namespace std;

class Student {
    static int count;          // declaration
public:
    Student() { count++; }    // increments on every object creation
    static int getCount() { return count; }
};
int Student::count = 0;      // definition + initialisation

int main() {
    Student s1, s2, s3;
    cout << "Students created = " << Student::getCount() << "\n";
    return 0;
}
```

OUTPUT

```
Students created = 3
```

2.6 Pointer to Class Member

DEFINITION

C++ allows taking the address of a (non-static) member itself, independent of any object, using a special **pointer-to-member** syntax. It is then applied to a specific object with `.*` or to a pointer-to-object with `->*`.

SYNTAX

```
// pointer to a data member
dataType ClassName::*ptr = &ClassName::member;
object.*ptr;                // access via object
objPtr->*ptr;                // access via pointer to object

// pointer to a member function
returnType (ClassName::*fptr)(params) = &ClassName::function;
(object.*fptr)(args);
```

KEY POINTS

- Rarely used in everyday code, but appears in exams and in advanced generic/callback programming.
- Different from an ordinary pointer: it stores an *offset/selector* for the member, not a fixed memory address, since the member's address depends on which object it is applied to.

PROGRAM

```
#include <iostream>
using namespace std;

class Demo {
public:
    int value = 42;
    void show() { cout << "value = " << value << "\n"; }
};

int main() {
    Demo d;
    int Demo::*pdata = &Demo::value;    // pointer to data member
    void (Demo::*pfunc)() = &Demo::show; // pointer to member function

    cout << "Via pointer-to-member data: " << d.*pdata << "\n";
    (d.*pfunc)();                        // call via pointer-to-member function
    return 0;
}
```

OUTPUT

```
Via pointer-to-member data: 42
value = 42
```

2.
7

Nested Classes

DEFINITION

A **nested class** is a class declared entirely inside another (enclosing) class. It is mainly a scoping/organisation tool — the nested class name is hidden inside the enclosing class's scope and must be qualified as `Outer::Inner` from outside.

SYNTAX

```
class Outer {
public:
    class Inner {                // nested class
    public:
        void show() { cout << "Inner\n"; }
    };
};

Outer::Inner obj;    // create an object of the nested class
```

KEY POINTS

- A nested class does not automatically get access to the enclosing class's private members, or vice-versa, unless explicitly made a friend.
- Useful for helper classes that are conceptually part of another class only (e.g. a `Node` class nested inside a `LinkedList` class).

PROGRAM

```
#include <iostream>
using namespace std;

class LinkedList {
public:
    class Node {                // nested helper class
    public:
        int data;
        Node *next;
        Node(int d) : data(d), next(nullptr) {}
    };
    Node *head = nullptr;
    void push(int v) {
        Node *n = new Node(v);
        n->next = head;
        head = n;
    }
    void print() {
        for (Node *t = head; t != nullptr; t = t->next)
            cout << t->data << " ";
        cout << "\n";
    }
};

int main() {
    LinkedList list;
    list.push(10); list.push(20); list.push(30);
    list.print();
    return 0;
}
```

OUTPUT

```
30 20 10
```

2. Class Initialization, Member-wise Initialization and Member-wise Assignment

DEFINITION

When an object is created from another object of the same class without a user-defined copy constructor, the compiler automatically performs **member-wise initialization** (also called the default copy constructor): each data member of the new object is initialised with the corresponding data member of the source object. Likewise, the built-in assignment operator performs **member-wise assignment** when no custom operator= is written.

SYNTAX

```
ClassName obj2 = obj1;           // member-wise (copy) initialization
ClassName obj2(obj1);           // same, explicit copy-constructor call
obj2 = obj1;                     // member-wise assignment (obj2 already exists)
```

KEY POINTS

- Member-wise copy is a **shallow copy**: if a data member is a pointer, only the pointer (address) is copied, not the pointed-to data — both objects then share the same dynamically allocated memory, which is dangerous (double free, dangling pointer).
- For classes that own a resource (e.g. heap memory), define an explicit copy constructor and operator= that perform a **deep copy**.

PROGRAM – MEMBER-WISE COPY (FINE FOR PLAIN DATA)

```
#include <iostream>
using namespace std;

class Point {
public:
    int x, y;
};

int main() {
    Point p1 = {3, 4};
    Point p2 = p1;           // member-wise initialization
    p2.x = 100;              // independent copy for plain members
    cout << "p1: (" << p1.x << ", " << p1.y << ") " << "
         << "p2: (" << p2.x << ", " << p2.y << ")\n";
    return 0;
}
```

OUTPUT

```
p1: (3,4) p2: (100,4)
```

2.
9

The Class Constructor

DEFINITION

A **constructor** is a special member function, with the same name as the class and no return type, that runs automatically when an object is created. Its job is to initialise the object's data members.

SYNTAX

```
class ClassName {
public:
    ClassName();           // default constructor
    ClassName(type1 a, type2 b); // parameterised constructor
    ClassName(const ClassName &obj); // copy constructor
};
```

KEY POINTS

- A **default constructor** takes no arguments (or all defaulted arguments). If you define no constructor at all, the compiler supplies an implicit, empty default constructor.
- A **parameterised constructor** takes arguments used to initialise members directly — preferably through a *member initializer list*: `ClassName(int a) : x(a) { }`, which is more efficient than assigning inside the body.
- A **copy constructor** `ClassName(const ClassName &obj)` initialises a new object as a copy of an existing one; the compiler generates one automatically (member-wise) if none is written.
- Constructors can be **overloaded** (multiple constructors with different parameter lists) — this is constructor overloading, a form of compile-time polymorphism.
- A constructor marked `explicit` prevents it from being used for implicit single-argument type conversions.

PROGRAM – DEFAULT, PARAMETERISED, AND OVERLOADED CONSTRUCTORS

```
#include <iostream>
using namespace std;

class Rectangle {
    double length, breadth;
public:
    Rectangle() : length(1), breadth(1) {} // default
    Rectangle(double l, double b) : length(l), breadth(b) {} // parameterised
    Rectangle(double side) : length(side), breadth(side) {} // overload: square
    double area() { return length * breadth; }
};

int main() {
    Rectangle r1; // calls default constructor
    Rectangle r2(4, 6); // calls parameterised constructor
    Rectangle r3(5); // calls the (double) overload -> square
    cout << r1.area() << " " << r2.area() << " " << r3.area() << "\n";
    return 0;
}
```

OUTPUT

```
1 24 25
```

PROGRAM – COPY CONSTRUCTOR (USER-DEFINED, DEEP COPY)

```
#include <iostream>
using namespace std;

class Marks {
    int *score;
public:
    Marks(int s) { score = new int(s); } // parameterised
    Marks(const Marks &m) { score = new int(*m.score); } // copy ctor: deep copy
    void set(int s) { *score = s; }
    int get() { return *score; }
    ~Marks() { delete score; }
};

int main() {
    Marks m1(80);
    Marks m2 = m1; // copy constructor called
    m2.set(95); // does NOT affect m1 (separate heap memory)
    cout << "m1 = " << m1.get() << ", m2 = " << m2.get() << "\n";
    return 0;
}
```

OUTPUT

```
m1 = 80, m2 = 95
```

2.10 The Class Destructor

DEFINITION

A **destructor** is a special member function, named `~ClassName`, with no return type and no parameters, that runs automatically when an object's lifetime ends (goes out of scope, or is `deleted`). Its job is to release resources the object acquired (especially dynamically allocated memory, open files, locks).

SYNTAX

```
class ClassName {
public:
    ~ClassName();           // destructor: no arguments, no return type, one per class
};

ClassName::~~ClassName() {
    // cleanup code, e.g. delete dynamically allocated members
}
```

KEY POINTS

- A class can have only **one** destructor (it cannot be overloaded, since it takes no parameters).
- If not written explicitly, the compiler supplies an implicit destructor that destroys each member but does not free any manually allocated heap memory — so any class that uses `new` needs its own destructor.
- Destructors run in reverse order of construction; for an array/container of objects, each element's destructor runs when it is destroyed.
- In inheritance (Unit III), a base class destructor should be `virtual` if the class will be deleted through a base pointer.

PROGRAM

```
#include <iostream>
using namespace std;

class Buffer {
    int *data;
    int size;
public:
    Buffer(int n) : size(n) {
        data = new int[n];
        cout << "Constructor: allocated " << n << " ints\n";
    }
    ~Buffer() {
        delete[] data;
        cout << "Destructor: memory released\n";
    }
};

int main() {
    {
        Buffer b(5);
        cout << "Using buffer...\n";
    } // b goes out of scope here -> destructor runs automatically
    cout << "After block\n";
    return 0;
}
```

OUTPUT

```
Constructor: allocated 5 ints
Using buffer...
Destructor: memory released
After block
```

2.11 Class Object Arrays

DEFINITION

Just as built-in types can form arrays, so can class objects: an array of objects allocates that many instances contiguously, calling the class's default constructor for each element (unless initialised individually).

SYNTAX

```
ClassName arr[size]; // default-constructed elements
ClassName arr[size] = { ClassName(a1), ClassName(a2) }; // explicit initialisation
```

KEY POINTS

- A class used in a default-constructed array **must** have an accessible default constructor (or every element must be explicitly initialised).
- Individual objects are accessed with the usual index operator: `arr[i].memberFunction()`.

PROGRAM

```
#include <iostream>
using namespace std;

class Student {
    int roll; float marks;
public:
    Student() : roll(0), marks(0) {}
    void set(int r, float m) { roll = r; marks = m; }
    void show() { cout << "Roll " << roll << ": " << marks << "\n"; }
};

int main() {
    Student arr[3]; // 3 default-constructed objects
    arr[0].set(1, 78.5);
    arr[1].set(2, 85.0);
    arr[2].set(3, 91.5);
    for (int i = 0; i < 3; i++) arr[i].show();
    return 0;
}
```

OUTPUT

```
Roll 1: 78.5
Roll 2: 85
Roll 3: 91.5
```

2.12 Function Overloading

DEFINITION

Function overloading lets several functions share the same name as long as their parameter lists differ in number or type (not merely return type). The compiler picks the matching version at compile time based on the arguments supplied — this is *compile-time* (static) polymorphism.

SYNTAX

```
returnType functionName(type1 a);
returnType functionName(type1 a, type2 b);
returnType functionName(otherType a);
```

KEY POINTS

- Overload resolution is based on the number, order and types of arguments — return type alone cannot distinguish two overloads.
- Constructor overloading (seen in Topic 2.9) is a special case of function overloading.
- Overloading differs from overriding: overloading is same name/different signature in the *same* scope; overriding (Unit III) involves a *derived* class redefining a base class's virtual function with the *same* signature.

PROGRAM

```
#include <iostream>
using namespace std;

int add(int a, int b) { return a + b; }
double add(double a, double b) { return a + b; }
int add(int a, int b, int c) { return a + b + c; }

int main() {
    cout << add(2, 3) << "\n";           // calls int version
    cout << add(2.5, 3.5) << "\n";       // calls double version
    cout << add(1, 2, 3) << "\n";       // calls 3-argument version
    return 0;
}
```

OUTPUT

```
5
6
6
```

2.13 Operator Overloading

DEFINITION

Operator overloading gives extra meaning to C++'s built-in operators (+, -, ==, <<, [] ...) when applied to objects of a user-defined class, so objects can be used as naturally as built-in types (e.g. adding two Complex numbers with +).

SYNTAX

```
// as a member function (left operand is the implicit object)
returnType operator op (parameters);

// as a friend / non-member function (needed when the left operand
// is not an object of this class, e.g. cout << obj)
returnType operator op (ClassName &obj, ...);
```

KEY POINTS

- Most operators can be overloaded; a few cannot be overloaded at all: :: . .* ?: sizeof.
- Overloading does not change an operator's precedence, associativity, or number of operands.
- Overload << and >> (stream insertion/extraction) as **friend** functions, since the left operand is a stream object, not the class itself.
- This is another form of (compile-time) polymorphism: the same operator symbol performs different actions depending on the operand types.

PROGRAM – OVERLOADING + AS A MEMBER FUNCTION

```
#include <iostream>
using namespace std;

class Complex {
    double re, im;
public:
    Complex(double r = 0, double i = 0) : re(r), im(i) {}
    Complex operator+(const Complex &c) { // member overload
        return Complex(re + c.re, im + c.im);
    }
    void show() { cout << re << " + " << im << "i\n"; }
};

int main() {
    Complex c1(2, 3), c2(4, 5);
    Complex c3 = c1 + c2; // calls operator+
    c3.show();
    return 0;
}
```

OUTPUT

```
6 + 8i
```

PROGRAM – OVERLOADING << AS A FRIEND FUNCTION (FOR COUT)

```
#include <iostream>
using namespace std;

class Point {
    int x, y;
public:
    Point(int a, int b) : x(a), y(b) {}
    friend ostream& operator<<(ostream &out, const Point &p);
};

ostream& operator<<(ostream &out, const Point &p) {
    out << "(" << p.x << ", " << p.y << ")";
    return out;          // enables chaining: cout << p1 << p2;
}

int main() {
    Point p(3, 4);
    cout << "Point = " << p << "\n";    // uses the overloaded <<
    return 0;
}
```

OUTPUT

```
Point = (3, 4)
```

KEY POINTS

- A comparison overload `bool operator==(const ClassName&)` is a popular exam question, often paired with sorting / STL use in Unit V.

Constructor Types — Quick Comparison

Constructor	When Called	Example
Default	No arguments supplied, e.g. Student s;	Student() {}
Parameterised	Arguments supplied at creation	Student(int r) : roll(r) {}
Copy	New object built from an existing one	Student(const Student &s;)
Overloaded	Several constructors, different signatures	Rectangle(), Rectangle(double), Rectangle(double,double)

2.14 Deep Dive — Constructors: Advantages, Disadvantages, Applications**Advantages, Disadvantages and Applications****ADVANTAGES**

- Guarantee an object is never left in an uninitialised, unpredictable state.
- Support multiple ways to build an object through overloading (default, parameterised, copy).
- Member initializer lists initialise data members directly (more efficient than assigning in the body, especially for const/reference members and class-type members).
- Enable automatic, compiler-driven object setup with zero extra calls needed from the programmer.

DISADVANTAGES

- Cannot return a value or report failure the way an ordinary function can (must use exceptions to signal construction failure).
- The compiler-generated default/copy constructor (member-wise copy) performs only a shallow copy, which is unsafe for classes owning heap memory.

- Overloading too many constructors can make a class harder to understand and maintain.

APPLICATIONS

- Initialising class invariants (e.g. ensuring a Fraction's denominator is never zero).
- Resource acquisition in the RAII (Resource Acquisition Is Initialization) idiom — opening a file/allocating memory in the constructor, releasing it in the destructor.
- Building objects from different kinds of input via overloaded/parameterised constructors (e.g. Rectangle(side) for a square, Rectangle(l,b) in general).

Line-by-Line Explanation

Line / Statement	Explanation
class Rectangle {	Begins the definition of a user-defined class named Rectangle.
double length, breadth;	Two private data members (default access in a class) to hold the rectangle's dimensions.
Rectangle() : length(1), breadth(1) {}	Default constructor: runs when an object is created with no arguments; the member initializer list sets both dimensions to 1 before the (empty) constructor body runs.
Rectangle(double l, double b) : length(l), breadth(b) {}	Parameterised constructor: overload taking two doubles, initialising length and breadth directly from the arguments.
Rectangle(double side) : length(side), breadth(side) {}	A further overload taking a single double, used to construct a square (both dimensions equal) — demonstrates constructor overloading.
double area() { return length * breadth; }	An ordinary member function computing the rectangle's area from its two data members.
Rectangle r1;	Creates r1 with no arguments; the compiler selects the zero-argument (default) constructor.
Rectangle r2(4, 6);	Creates r2 with two arguments; the compiler selects the two-double-parameter constructor.
Rectangle r3(5);	Creates r3 with one argument; the compiler selects the single-double-parameter (square) constructor — this is overload resolution at work.

COMMON MISTAKES / DEBUGGING TIPS

- Writing a copy constructor as `ClassName(ClassName obj)` (by value) instead of `ClassName(const ClassName &obj)` — pass-by-value would itself try to copy-construct the argument, causing infinite recursion.
- Relying on the compiler-generated (member-wise) copy constructor for a class that holds a raw pointer to heap memory — this produces a shallow copy where two objects share and later both try to delete the same memory (double free).
- Forgetting that constructors with a **single** parameter can be used by the compiler for *implicit* type conversions, unless marked `explicit` — this can cause confusing, unintended conversions.
- Initialising members in the constructor *body* with `=` instead of the member initializer list — this works for simple types but is required (not just preferred) for `const` members, reference members, and members without a default constructor.

2.15 Deep Dive — Destructors and the Rule of Three

If a class defines (or needs) any **one** of a custom destructor, a custom copy constructor, or a custom copy-assignment operator, it almost always needs all three — this guideline is the "**Rule of Three**". The reasoning: any class that manages a resource (typically via a raw pointer obtained with `new`) needs a destructor to free it; but the compiler-generated copy constructor and `operator=` would still copy the raw pointer itself (shallow copy), leaving two objects pointing at — and eventually both destructing — the same memory.

```

#include <iostream>
using namespace std;

class SafeArray {
    int *data;
    int size;
public:
    SafeArray(int n) : size(n) { data = new int[n]; }           // 1. constructor
    ~SafeArray() { delete[] data; }                             // 2. destructor
    SafeArray(const SafeArray &other) : size(other.size) {      // 3. copy constructor (deep copy)
        data = new int[size];
        for (int i = 0; i < size; i++) data[i] = other.data[i];
    }
    SafeArray& operator=(const SafeArray &other) {               // 4. copy assignment (deep copy)
        if (this == &other) return *this;                      // guard against self-assignment
        delete[] data;
        size = other.size;
        data = new int[size];
        for (int i = 0; i < size; i++) data[i] = other.data[i];
        return *this;
    }
};

```

COMMON MISTAKES / DEBUGGING TIPS

- Omitting the self-assignment guard (`if (this == &other) return *this;`) in `operator=` — without it, `obj = obj;` deletes the data before copying from itself, corrupting the object.
- Defining a deep-copy constructor but forgetting `operator=` (or vice versa) — the missing one silently falls back to the dangerous compiler-generated shallow version.

2.16 Comparison — Constructor vs Destructor

Point	Constructor	Destructor
Name	Same as the class name	Class name prefixed with ~
Return type	None (not even void)	None (not even void)
Parameters	Can take parameters; can be overloaded	Takes no parameters; cannot be overloaded
Count per class	Several (overloaded) allowed	Exactly one per class
Called	Automatically, when an object is created	Automatically, when an object is destroyed
Typical job	Initialise data members / acquire resources	Release resources / cleanup

2.17 Comparison — Function Overloading vs Operator Overloading

Point	Function Overloading	Operator Overloading
What is reused	An ordinary function name	A built-in operator symbol (+, ==, << ...)
Trigger	Different parameter list	Different operand types (one of which is a class)
Syntax	<code>returnType name(params)</code>	<code>returnType operator symbol(params)</code>
Can change precedence/arity?	N/A	No — precedence and number of operands stay fixed
Example	<code>add(int,int) / add(double,double)</code>	<code>Complex operator+(Complex)</code>

2.18 Deep Dive — Dry Run: Object Counter using Static Members (Topic 2.5)

TRACE TABLE

Statement	Student::count before	Action	Student::count after
Student s1;	0	Constructor runs, count++	1
Student s2;	1	Constructor runs, count++	2
Student s3;	2	Constructor runs, count++	3
Student::getCount()	3	Returns the single shared static value	3 (printed)

Because count is static, all three objects share and update the SAME memory location — there are not three separate counters.

2.19 Deep Dive — Friend Functions: Advantages, Disadvantages, Applications

Advantages, Disadvantages and Applications

ADVANTAGES

- Allow a carefully chosen external function to access private/protected data without making that data fully public.
- Enable operators such as << and >> to be overloaded naturally (their left operand is a stream, not the class).
- Allow one function to operate efficiently on the private data of two different classes at once.

DISADVANTAGES

- Breaks strict encapsulation — a friend function is not bound by the class's access control.
- Friendship is not inherited and not transitive, which can surprise programmers expecting OOP-style access rules.
- Overuse makes it harder to reason about which external code can modify an object's internals.

APPLICATIONS

- Overloading cout << obj and cin >> obj for custom classes (Topic 2.13).
- Comparator functions that need private data from two different class types.
- Matrix/vector libraries where helper operations need tight, efficient access to internal storage.

2.
20

Examination Preparation — Chapter 2

NOTE ON QUESTION SOURCE

- The questions below are **expected / practice questions** prepared from the official RCUB syllabus for this chapter, in the common style of university exam papers.
- They are **not** claimed to be verified previous-year question papers — cross-check against your department's official previous-year papers for the exact verified questions, if available.

A. Two-Mark Questions and Answers

Q1. [2 marks] Define a constructor and a destructor.

A constructor is a special member function, named like the class, called automatically when an object is created, used to initialise it. A destructor, named ~ClassName, is called automatically when an object is destroyed, used to release resources.

Q2. [2 marks] What is the default constructor?

A constructor that takes no arguments (or has all parameters defaulted); the compiler supplies one automatically if no constructor is written at all.

Q3. [2 marks] What is the this pointer?

An implicit pointer available inside every non-static member function, pointing to the object on which the function was called.

Q4. [2 marks] What is a static data member?

A data member shared by all objects of a class — only one copy exists regardless of how many objects are created.

Q5. [2 marks] Can a destructor be overloaded? Why or why not?

No. A destructor takes no parameters, so there is no way to distinguish multiple versions — a class can have only one destructor.

Q6. [2 marks] What is a friend function?

A non-member function explicitly granted access to a class's private and protected members, declared inside the class with the friend keyword.

Q7. [2 marks] What is member-wise initialization?

The compiler-generated default behaviour of the copy constructor: each data member of the new object is initialised from the corresponding member of the source object (a shallow copy).

Q8. [2 marks] What is the scope resolution operator used for in classes?

:: is used to define a member function outside the class body (ClassName::function) and to qualify a static member (ClassName::member).

Q9. [2 marks] What is operator overloading?

Giving extra meaning to a built-in operator (such as +, ==, <=) when applied to objects of a user-defined class.

Q10. [2 marks] Why must operator<< be overloaded as a friend function, not a member?

Because the left operand of << is a stream object (cout), not an object of the class, so it cannot be a member function of the class, which requires the left operand to be the implicit object.

Q11. [2 marks] What is a nested class?

A class declared entirely inside another (enclosing) class, mainly for scoping/organisational purposes (e.g. Node nested inside LinkedList).

Q12. [2 marks] What is function overloading?

Defining multiple functions with the same name but different parameter lists (number or type), resolved by the compiler at compile time.

Q13. [2 marks] What is the difference between a class and a struct in C++?

The only structural difference is default access: class members default to private, struct members default to public.

Q14. [2 marks] What is a parameterised constructor?

A constructor that takes one or more arguments, used to initialise an object's data members with specific values at creation.

Q15. [2 marks] Why should a copy constructor take its parameter by const reference?

To avoid infinite recursion (pass-by-value would itself require copy-construction) and to prevent the constructor from modifying the source object.

B. Five-Mark Questions and Answers

Q1. [5 marks] Explain constructors and their types with example programs.

A constructor is a special member function with the same name as the class and no return type, invoked automatically when an object is created, to initialise its data members. A **default constructor** takes no arguments (Rectangle() : length(1), breadth(1) {}); a **parameterised constructor** accepts arguments to initialise members directly, ideally via a member initializer list (Rectangle(double l, double b) : length(l), breadth(b) {}); a **copy constructor** (ClassName(const ClassName &obj;)) builds a new object as a copy of an existing one, and should perform a deep copy if the class owns heap memory. Constructors can be overloaded, as shown in Topic 2.9's Rectangle example, where the compiler picks Rectangle(), Rectangle(double,double) or Rectangle(double) based on the number/type of arguments supplied at the call site — this is compile-time polymorphism.

Q2. [5 marks] Explain the this pointer with two practical use cases and example code.

The this pointer is an implicit parameter available inside every non-static member function, pointing to the object the function was invoked on; it is how the same function body can operate on different objects' individual data. Use case 1: resolving a name clash between a parameter and a data member of the same name, e.g. void setXY(int x, int y) { this->x = x; this->y = y; }. Use case 2: enabling method chaining by returning *this from a member function, so calls like builder.add("a").add("b") can be written fluently, each call returning a reference to the same object for the next call to act on.

Q3. [5 marks] Discuss static data members and static member functions with a program.

A static data member is shared across all objects of a class — only one copy exists in memory no matter how many objects exist — declared inside the class but defined (and usually initialised) exactly once outside it, e.g. int Student::count = 0;. A static member function can be called using the class name alone (Student::getCount()), without needing any object, and can access only static members since it has no this pointer. The worked Topic 2.5 program demonstrates this by incrementing a static count in every constructor call and showing that three created objects (s1, s2, s3) all share and correctly reflect the same incrementing counter.

Q4. [5 marks] Explain friend functions with their characteristics and an example.

A friend function is a non-member function explicitly granted access to a class's private/protected members; it is declared inside the class using the friend keyword but defined outside like an ordinary function, so it is not called using the dot operator and has no this pointer. Friendship is granted by the class (not requested by the function), is not inherited by derived classes, and is not symmetric/mutual between two classes. Typical applications are operator overloading (especially << and >>) and functions needing access to the private data of two different classes at once, as shown by the bigger(const Box&, const Box&) example in Topic 2.3, which compares the private volume of two Box objects.

Q5. [5 marks] Explain operator overloading in C++, including rules and restrictions, with a worked example.

Operator overloading gives extra meaning to a built-in C++ operator when at least one operand is a user-defined class object, letting objects be used as naturally as built-in types. It can be implemented as a member function (the left operand is the implicit object, e.g. Complex operator+(const Complex&)) or as a friend/non-member function (needed when the left operand is not a class object, e.g. ostream& operator<<(ostream&, const Point&)). Restrictions: an operator's precedence, associativity and number of operands cannot be changed, and a small fixed set of operators (::, ., *, ?, sizeof) can never be overloaded. The Topic 2.13 Complex class example overloads + as a member function to add two complex numbers, and overloads << as a friend function so a Point object can be printed directly with cout.

Q6. [5 marks] Discuss class object arrays and member-wise initialization/assignment with suitable examples.

An array of class objects allocates that many instances contiguously, calling the default constructor for each element unless explicitly initialised — this requires the class to have an accessible default constructor. Member-wise initialization is the compiler's automatic behaviour when one object is initialised from another without a user-defined copy constructor: each data member is copied individually from the source. This becomes a shallow copy when a member is a pointer, which is safe for plain-data classes (such as the Point example in Topic 2.8) but dangerous for classes owning heap memory, where a user-defined deep-copy constructor and operator= are required instead (see the Rule of Three, Topic 2.15).

C. Ten-Mark / Long-Answer Questions

Q1. [10 marks] Explain classes and objects in C++ in detail: definition, access specifiers, data members, member functions, and the dot operator, with a complete program.

A class is a user-defined blueprint that groups data members (state) and member functions (behaviour) under one name, with access control via private, protected and public specifiers; private is the default in a class. Private/protected members can be reached only from inside the class (protected also from derived classes), while public members form the class's external interface, accessed using the dot operator on an object (obj.member) or the arrow operator through a pointer (ptr->member). Member functions may be defined inline inside the class body, or outside it using the scope resolution operator (ClassName::function). A complete answer should present the Student class from Topic 2.1 — private rollNo/name/marks, public setData()/display() — explain why the data is private (encapsulation: controlled access) and trace what happens when s.setData(101,"Asha",88.5); s.display(); executes.

Q2. [10 marks] Discuss constructors and destructors comprehensively, covering all constructor types, the Rule of Three, and construction/destruction order, with example programs.

Constructors (same name as the class, no return type) initialise an object automatically on creation; destructors (~ClassName, no return type, no parameters, cannot be overloaded) run automatically when an object's lifetime ends, to release resources. Constructor types: default (no arguments), parameterised (arguments initialise members, preferably via a member initializer list), and copy (ClassName(const ClassName&), building a new object from an existing one). The **Rule of Three** states that a class needing a custom destructor (because it manages a resource, e.g. heap memory via new) almost always also needs a custom copy constructor and copy-assignment operator, since the compiler-generated versions perform only a shallow copy, which would cause two objects to share (and later both free) the same memory — demonstrated by the SafeArray class in Topic 2.15, including the essential self-assignment guard inside operator=. For class object arrays and base/derived hierarchies, construction always proceeds in a fixed order and destruction happens in the exact reverse order.

Q3. [10 marks] Explain operator overloading and function overloading in detail, with the rules for each, restrictions, and complete programs showing both.

Function overloading lets several functions share a name provided their parameter lists differ in number or type (return type alone cannot distinguish overloads); resolved entirely at compile time, it is a form of static polymorphism, demonstrated by the three add() overloads in Topic 2.12. Operator overloading extends this idea to C++'s built-in operators: a class can define what + or << mean when applied to its objects, implemented either as a member function (left operand is the implicit object, e.g. Complex::operator+) or as a friend function (needed when the left operand is not the class itself, e.g. operator<< with cout as the left operand). A small, fixed set of operators can never be overloaded (::, ., *, ?, sizeof), and overloading never changes an operator's precedence, associativity, or arity. A thorough answer includes both the Complex operator+ example and the Point operator<< friend-function example from Topic 2.13, explaining why each is implemented the way it is.

Q4. [10 marks] Discuss static class members, the this pointer, pointer-to-member, and nested classes as mechanisms for organising class-level behaviour, with programs.

A static data member is shared by every object of a class (one copy total), declared inside the class and defined once outside it; a static member function can be called via the class name alone and can access only static data, since it lacks a this pointer. The this pointer, present in every non-static member function, points to the specific object a call was made on, enabling disambiguation of same-named parameters/members and method chaining via *this. Pointer-to-member syntax (dataType ClassName::*ptr = &ClassName::member;) lets code refer to a member independent of any specific object, later applied to an object with .* or ->*, used in advanced generic and callback-style programming. A nested class (declared entirely inside another class, e.g. a Node class nested inside LinkedList) is primarily an organisational/scoping tool, hiding a helper type inside the scope of the class it conceptually belongs to, accessed from outside as Outer::Inner.

Q5. [10 marks] Explain, with programs, how encapsulation and controlled access are achieved through access specifiers, friend functions, and const correctness in class design.

Encapsulation is achieved by making data members private and exposing a controlled public interface (getters/setters), as shown by the Student class in Topic 2.1, so external code cannot directly corrupt an object's internal state. Friend functions are a deliberate, limited exception to this rule: a specific external function (or class) is granted access to private/protected members, used sparingly for cases like operator<<, or functions comparing the private data of two different classes (Topic 2.3's bigger() example). const correctness complements encapsulation: a const member function promises not to modify the object it is called on, and a function parameter such as void printStudent(const Student &s;) combines pass-by-reference efficiency (Topic 1.10) with a compiler-enforced guarantee that the function cannot alter the caller's object — together, access specifiers, selective friendship, and const correctness let a class designer precisely control what external code can and cannot do to an object.

D. Important Programming Questions

PRACTICE: WRITE THE FULL PROGRAM

- Write a C++ program to define a Complex class and overload +, - and == operators for it.
- Write a C++ program with a static data member that counts how many objects of a class have been created and destroyed (increment in constructor, decrement in destructor).
- Write a C++ program demonstrating a shallow-copy bug using the compiler-generated copy constructor with a class that holds a raw pointer, then fix it with a user-defined deep-copy constructor.
- Write a C++ program that overloads << and >> as friend functions so objects of a Student class can be read with cin and printed with cout directly.
- Write a C++ program using the this pointer to implement method chaining for a simple StringBuilder-like class.

E. Output Prediction Questions

Q1. Predict the output:

```
#include <iostream>
using namespace std;

class Counter {
    static int count;
public:
    Counter() { count++; }
    static int get() { return count; }
};
int Counter::count = 0;

int main() {
    Counter a, b;
    { Counter c; }
    cout << Counter::get();
    return 0;
}
```

ANSWER

3

Explanation: Every Counter object created increments the single shared static count, regardless of scope or whether it later goes out of scope — a, b and the temporary c inside the inner block all increment it once each, giving 3.

Q2. Predict the output:

```
#include <iostream>
using namespace std;

class Box {
    int side;
public:
    Box(int s) : side(s) {}
    Box(const Box &b) { side = b.side + 1; cout << "copy ctor\n"; }
};

int main() {
    Box b1(5);
    Box b2 = b1;
    return 0;
}
```

ANSWER

copy ctor

Explanation: Box b2 = b1; invokes the copy constructor (not operator=, since b2 is being created, not reassigned), printing "copy ctor"; no output of side is requested so only that line prints.

Q3. Predict the output:

```
#include <iostream>
using namespace std;

class Demo {
public:
    void show() { cout << "A"; }
};

class Demo2 : public Demo {
public:
    void show() { cout << "B"; }
};

int main() {
    Demo2 d;
    d.show();
    d.Demo::show();
    return 0;
}
```

ANSWER

BA

Explanation: d.show() calls Demo2's own show() ("B"); d.Demo::show() explicitly calls the base class version using the scope resolution operator, printing "A" — together "BA".

F. Find-the-Error Questions

Q1. Identify and correct the error:

```
#include <iostream>
using namespace std;

class Box {
    int side;
public:
    Box(Box b) { side = b.side; }    // ERROR
};
```

EXPLANATION

A copy constructor must take its parameter by reference, not by value: `Box(Box b)` causes infinite recursion, because passing `b` by value itself requires invoking the copy constructor again to make the parameter copy. Fix: `Box(const Box &b;)`.

Q2. Identify and correct the error:

```
#include <iostream>
using namespace std;

class Demo {
public:
    ~Demo() { cout << "destroyed"; }
    ~Demo() { cout << "again"; }    // ERROR
};
```

EXPLANATION

A class can have only ONE destructor — destructors cannot be overloaded, since they take no parameters to distinguish them. This is a compile-time error.

Q3. Identify and correct the error:

```
#include <iostream>
using namespace std;

class Counter {
    static int count;    // ERROR (if nothing else follows)
public:
    Counter() { count++; }
};

int main() { Counter c; }
```

EXPLANATION

A static data member must be defined (given storage) exactly once outside the class, e.g. `int Counter::count = 0;` — without this line, the program fails to link ("undefined reference to `Counter::count`").

G. Multiple Choice Questions (with explanations)

1. What is the default access specifier for members of a class?

(A) public (B) protected (C) private ✓ (D) friend

Answer: (C). Members of a class default to private (unlike a struct, which defaults to public).

2. Which special member function has the same name as the class and no return type?

(A) Destructor (B) Constructor ✓ (C) Friend function (D) Static function

Answer: (B). A constructor shares the class's name and has no return type (not even void); it runs automatically on object creation.

3. How many destructors can a class have?

(A) As many as needed (overloaded) (B) Exactly one ✓ (C) Zero (D) Two (one default, one parameterised)

Answer: (B). A destructor takes no parameters, so it cannot be overloaded — a class may have only one.

4. Which keyword is used to grant a non-member function access to a class's private data?

(A) static (B) virtual (C) friend ✓ (D) private

Answer: (C). *friend grants a specific external function or class access to private/protected members.*

5. What does the `this` pointer refer to inside a non-static member function?

(A) The class itself (B) The object the function was called on ✓ (C) The base class (D) A static copy of the object

Answer: (B). *this points to the specific object instance a non-static member function was invoked on.*

6. Which of these can be used to call a static member function?

(A) Only through an object (B) Only through a pointer (C) The class name alone (`ClassName::function()`) ✓ (D) It cannot be called directly

Answer: (C). *A static member function belongs to the class, not any object, so it can (and usually is) called as `ClassName::function()`, with no object required.*

7. Which operator must be overloaded as a friend function rather than a member function, and why?

(A) `operator+`, because it is binary (B) `operator<<`, because the left operand is a stream object, not the class ✓ (C) `operator=`, because it must be a member (D) `operator[]`, because arrays need it

Answer: (B). *`cout << obj` requires the left operand to be the stream (`ostream&`), which cannot be the implicit "this" of the class, so it must be a non-member (friend) function.*

8. What kind of copy does the compiler-generated copy constructor perform?

(A) Deep copy (B) Shallow (member-wise) copy ✓ (C) No copy at all (D) Copy only of static members

Answer: (B). *Without a user-defined copy constructor, the compiler performs a shallow, member-wise copy, which is unsafe when a member is a raw pointer to heap memory.*

9. A class with a user-defined destructor managing heap memory, by the Rule of Three, should also define:

(A) Only a default constructor (B) A copy constructor and copy-assignment operator ✓ (C) A friend function (D) A static member

Answer: (B). *The Rule of Three says such a class needs a custom copy constructor AND copy-assignment operator too, to avoid the dangerous default shallow copy.*

10. Which is NOT a valid constructor type?

(A) Default constructor (B) Parameterised constructor (C) Copy constructor (D) Virtual constructor ✓

Answer: (D). *There is no such thing as a "virtual constructor" in C++ — constructors cannot be declared virtual (though destructors can be, see Unit III).*

H. Viva-Voce Questions

BE READY TO ANSWER VERBALLY, IN YOUR OWN WORDS

- Why can't a constructor have a return type, not even void?
- What happens if you don't define any constructor for a class?
- What is the practical risk of relying on the default (compiler-generated) copy constructor?
- Why must the copy constructor's parameter be a const reference?
- What is the self-assignment problem in `operator=`, and how do you guard against it?
- Why can destructors not be overloaded?
- Is a friend function a member of the class? How is it called?
- Can a static member function access non-static data members directly? Why not?
- What is the difference between a member function defined inside vs outside the class body?
- Why is `operator<<` typically overloaded as a friend, not a member, function?
- What is the Rule of Three, and why does it matter?
- What is the difference between function overloading and operator overloading?
- What is a nested class used for?
- Explain pointer-to-member syntax briefly, and why it differs from an ordinary pointer.
- Give one example where a class array of objects could fail to compile, and explain why.

I. Chapter 2 — Revision Notes (Must-Remember Points)

TOP 10 EXAM-CRITICAL POINTS

- Class members default to private; struct members default to public — that is the only structural difference.
- Constructor: same name as class, no return type, can be overloaded, runs on object creation.
- Destructor: ~ClassName, no return type, no parameters, cannot be overloaded, runs on object destruction, in reverse order of construction.
- Copy constructor parameter must be a const reference to avoid infinite recursion.
- Rule of Three: custom destructor + custom copy constructor + custom operator= should come as a set when a class owns a resource.
- static data member: one copy shared by all objects; must be defined once outside the class.
- this pointer: implicit, points to the current object, used for disambiguation and method chaining (*this).
- friend function: granted access explicitly, not a member, not inherited, not mutual.
- operator<< / operator>> are overloaded as friend functions because the left operand is a stream, not the class.
- Operator overloading never changes precedence, associativity, or number of operands; a few operators can never be overloaded (:: . * ?: sizeof).